

Verified Optimizations to Flock’s RS Prover

measurements on Apple M1 Max, single-threaded; zerocheck at 2^{18} , PCS open at 2^{16}

BLAKE3 compressions

branch `snark-fast-improvements`

August 2026

Abstract

Seven changes to the RS (additive-NTT) zerocheck path, each kept only after a paired, order-alternating A/B measurement with a decisive sign test. Together they reduce zerocheck round 1 from 1477 ms to 1205 ms (-18.4%), round 2 from 433 to 312 ms (-28%), the rounds-3+ tail from 455 to 307 ms (-33%), and whole-proof time by $\approx 11\%$ single-threaded. Three are ideas ported from an independently optimized fork of this prover (the “challenge” tree), re-derived and re-implemented rather than copied; three originated here. The recurring themes: reductions in fields of characteristic two are $\mathbb{F}_2$ -linear and can be deferred almost arbitrarily; work that a circuit fixes structurally can be skipped with a one-word guard; and on Apple silicon the boundary between the vector and general register files costs more than most instruction counts. A later section extends the record to the PCS open, whose dominant pass loses its per-slot field multiply to the same $\mathbb{F}_2$ -linearity argument (§8).

1 Setting and notation

Fields. $\mathbb{F}_{2^8} = \mathbb{F}_2[x]/(x^8 + x^4 + x^3 + x + 1)$ and the GHASH field $\mathbb{F}_{2^{128}} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$, with the ring embedding $\varphi : \mathbb{F}_{2^8} \hookrightarrow \mathbb{F}_{2^{128}}$. Write $\alpha := \varphi(x)$ and $\gamma := X$.

The witness is $z \in \mathbb{F}_2^{2^m}$; at the benchmark shape $m = 32$ (2^{18} BLAKE3 compressions, $k_{\log} = 14$ bits per block). After the univariate skip, round 1 of the zerocheck views the m index bits as

$$\left[\underbrace{\lambda}_{6 \text{ (univariate)}} \mid \underbrace{s}_{3 \text{ (small)}} \mid \underbrace{b}_{4 \text{ (medium)}} \mid \underbrace{t}_{m-13 \text{ (rest)}} \right],$$

and must emit, for each of the 64 points λ of the skip domain, the quadratic message $P^{AB}(\lambda)$ and the linear message $P^C(\lambda)$, plus a pair of *banks* (the ring-switch helper $\hat{s}_{v,C}$) that keep the small parity dimension unfolded.

The protocol pins the small and medium challenges to *friendly* values: $r_{6+i} = \varphi(v_i)$ with $v = (0\text{x}F7, 0\text{x}53, 0\text{x}B5)$, and $\beta_i = \gamma^{2^{i-1}}/(1 + \gamma^{2^{i-1}})$. These make the eq weights geometric:

$$\prod_{i=0}^2 \text{eq}(r_{6+i}, K_i) = C_s \alpha^K, \quad \prod_{i=1}^4 \text{eq}(\beta_i, b_i) = \gamma^b / D, \quad (1)$$

with $K = \sum K_i 2^i$, $C_s = \prod_i (1 + r_{6+i}) = \varphi(0\text{x}1C)$, and $D = \prod_i (1 + \gamma^{2^{i-1}})$. The baseline kernels exploit (1) through lookup tables; several optimizations below exploit it in the opposite direction, replacing tables by folds at the actual challenge values.

Measurement protocol. All numbers are single-threaded at the shape above. Each verdict is a paired A/B: both arms built once, one discarded warm-up each, then eight pairs with the *order alternating within each pair* (a fixed order is biased by thermal drift by roughly the size of the effects measured here). Phase times are the minimum over the ~ 5 zerocheck invocations within one run. A change is kept only on a decisive sign test; two of the six wins were 8/8 with disjoint ranges. Measure only on AC power with the battery above $\sim 60\%$: low battery caps frequencies, and fast-charging a nearly empty battery is worse.

2 Deferred reduction in vector registers

For $u \in \mathbb{F}_2^{256}$ (an unreduced carry-less product) let $R(u) \in \mathbb{F}_{2^{128}}$ be reduction mod $X^{128} + X^7 + X^2 + X + 1$. R is $\mathbb{F}_2$ -linear, so for any products $u_i = a_i \odot b_i$ (unreduced),

$$\sum_i R(u_i) = R(\sum_i u_i),$$

and a sum of n field products needs one reduction, not n . The baseline already used this identity, but held the 256-bit accumulator as a four-word struct in *general* registers: each product paid six lane extracts to leave the vector file, and each accumulate was four scalar XORs.

The fix is representational, not algorithmic: keep the accumulator as two 128-bit vector registers. The unreduced product uses Karatsuba,

$$(a_l + a_h X^{64})(b_l + b_h X^{64}) : \quad ll = a_l b_l, \quad hh = a_h b_h, \quad cross = (a_l + a_h)(b_l + b_h) + ll + hh,$$

three PMULLs instead of the schoolbook four; accumulation is two vector XORs; the four extracts are paid once per worker chunk on the way out, and the existing scalar reducer is reused unchanged.

Measured: round-2 phase 433.1 $\rightarrow$ 375.2 ms (−13.4%).

3 Structurally fixed rows of the b operand

Round 1’s dominant kernel transforms the operands a, b through a table T implementing the inverse-NTT composition; T is $\mathbb{F}_2$ -linear, so $T(0) = 0$. A census of the packed BLAKE3 witness (256 word positions per block $\times$ 256 blocks $\times$ 3 independent witnesses) shows the circuit pins 38 of 256 eight-byte K -rows of b regardless of the inputs, taking only three values:

$$\underbrace{0\text{xff}^8}_{22 \text{ positions (const-one wires)}}, \quad \underbrace{0^8}_{15 \text{ positions (structural zeros)}}, \quad \text{one mixed word.}$$

For a zero row the entire row’s work vanishes: $db = T(0) = 0$, hence $y = da \odot db = 0$ contributes nothing to any accumulator. The kernel gains a single guard:

1: **if** $\text{load}_{64}(b_row) = 0$ **then return** $\triangleright$ skips all 64 table loads and 4 $\mathbb{F}_{2^8}$ multiplies

No census data ships and no positions are tracked: the guard is exact for any witness, and a disagreeing witness falls through to the generic path. (The all-ones rows were also tried: constant-folding them saves only the b half of a row’s work, and halving a row’s loads halves its memory-level parallelism, returning 6 ms where 27 were predicted. Reverted; whole-row elimination is what pays.)

Measured: round 1 −42.4 ms (−2.9%), 8/8.

4 Instruction-level parallelism in the gather drain

The drain accumulates, per output lane, three XOR chains of depth $n_{\text{med}} = 16$ (one per bank), each link a table gather feeding an XOR. The gathers are independent but the chains are serial, so one lane exposes three dependency chains to the out-of-order engine. Processing two lanes per iteration doubles that to six with no change in total work:

```

1: for  $\ell = 0, 2, 4, \dots, 62$  do
2:    $u_0, u_1, u_2 \leftarrow 0$ ;  $v_0, v_1, v_2 \leftarrow 0$  ▷ banks for lanes  $\ell$  and  $\ell+1$ 
3:   for  $b = 0 \dots n_{\text{med}} - 1$  do
4:      $u_j \oplus = \text{convert}[b][\cdot]$ ;  $v_j \oplus = \text{convert}[b][\cdot]$  ▷ 6 independent chains in flight
5:   end for
6: end for

```

This targets latency, not throughput: an attempt to shrink the gather *table* ($64 \rightarrow 8$ KiB, doubling gather count) had cost 3.6% — the table already fit M1’s 128 KiB L1D, establishing that the drain is bound by gather count and chain depth, not footprint.

Measured: round 1 -63.1 ms (-4.3%), 8/8. *Combined with Section 3:* $1477.3 \rightarrow 1403.1\text{ ms}$ (-5.0%).

5 Unreduced accumulation with multiplicative weight splitting

The prep kernel computes, per K -row ($K \in \{0, \dots, 7\}$) and 16-lane block, $y_K = a_K \odot b_K \in \mathbb{F}_{2^8}$ and accumulates $\sum_K x^K y_K$ in 16-bit lanes, reducing once at the end. The baseline computed each y_K *reduced*: two PMULLs for the raw product plus a reduction costing four more, then a widening shift by K — six PMULLs per row-block for a reduction that the final fold makes redundant.

Let u_K be the *unreduced* 15-bit product. Then $\sum_K x^K u_K \equiv \sum_K x^K y_K \pmod{p_8}$, but $x^K u_K$ overflows 16 bits for $K \geq 2$. Split the weight so every factor lands where it is free:

$$x^K = \underbrace{(x^4)^{[K \geq 4]}}_{\text{choice of gather table}} \cdot \underbrace{(x^2)^{[(K/2) \text{ odd}]}}_{\text{byte-wise doubling of the reduced operand}} \cdot \underbrace{x^{K \bmod 2}}_{\text{in-lane shift}}.$$

The x^4 factor costs nothing per element: a second table image with every byte pre-multiplied by x^4 scales each gathered XOR-sum by x^4 , by linearity of T . The x^2 factor is a six-instruction shift-and-conditional-XOR on the already-reduced 8-bit operand (deg still ≤ 7). The residual shift is one vector instruction and raises term degree to at most 15.

Algorithm 1 Row-block accumulate, before and after

```

1: before:  $acc \oplus = \text{shift}_K(\text{reduce}(\text{pmull}(da, db)))$  ▷ 6 PMULL
2: after:  $da \leftarrow [K \geq 4] ? \text{gathered from } x^4 \text{ table : } da$ ;  $da \leftarrow [(K/2) \bmod 2] ? x^2 \cdot da : da$ 
3:    $acc \oplus = \text{shift}_{K \bmod 2}(\text{pmull}(da, db))$  ▷ 2 PMULL

```

Correctness needs the final reducer to be exact to degree 15, one beyond its documented “ ≤ 14 ”; both the scalar and vector reducers were verified exhaustively over all 2^{16} inputs (tests now permanent). Gather traffic is unchanged apart from the second 16 KiB table image.

Measured: round 1 $1401.4 \rightarrow 1273.4\text{ ms}$ (-9.1%), 8/8, *every pair in* $[-8.7\%, -10.1\%]$. *The reference implementation’s attribution predicted* 100–140 ms; *measured* 128.

6 The C-side message as a fold of the lincheck stripe

In the fast hash setups $C = I$, so $\hat{c} = \hat{z}$ and everything round 1 emits on the C side is *linear* in the witness. Concretely the two banks are, for parity $p \in \{0, 1\}$ and lane λ ,

$$\text{bank}_p[\lambda] = \sum_{K \equiv p \pmod{2}} \alpha^K \sum_b \frac{\gamma^b}{D} \sum_t \text{eq}(r_{\geq 13}, t) z[\lambda, K, b, t], \quad (2)$$

a multilinear evaluation — which the baseline nevertheless computed with the quadratic side’s machinery: a bit transpose of every 8192-bit window plus 32 of the drain’s 48 gathers per lane.

By (1), a *true* multilinear fold at the actual pinned challenges reproduces the weights in (2) exactly. The prover already materializes the *lincheck stripe*, a repacking of z indexed $[i_{\text{inner}} | i_{\text{outer}}]$, for the later lincheck phase. So:

- 1: $v \leftarrow \text{fold_stripe_outer}(z_{\text{stripe}}, \text{eq}(r_{k_{\log}..m}))$ $\triangleright$ the one $O(2^m)$ pass; same kernel lincheck uses
- 2: **for** $d = k_{\log} - 1$ **downto** 7 **do** $\triangleright$ data halves each round: $2^{14} \rightarrow 2^7$ elements
- 3: $v[i] \leftarrow v[i] + r_d(v[i] + v[i + 2^d])$
- 4: **end for**
- 5: $\text{bank}_p[\lambda] \leftarrow \frac{\text{eq}(r_6, p)}{C_s} v[p \cdot 64 + \lambda]$ $\triangleright C_s = (1+r_6)(1+r_7)(1+r_8)$, from r itself

Dimension 6 (the parity) and dimensions 0.5 (the lanes) are kept unfolded; the constant in step 3 undoes the two eq factors the partial fold applied, derived from the identity $Y_p = (C_s/\text{eq}(r_6, p)) \text{bank}_p$ obtained by factoring (1) over the folded dimensions. With the banks gone from the drain, the workers run AB-only: the per-window bit transpose is deleted and the drain issues one gather per (lane, b) instead of three. Equivalence was proven at two levels before wiring: banks bit-identical to the drain’s, and all three entry outputs identical, dense and padded.

Measured: round 1 1277.5 $\rightarrow$ 1204.7 ms (−5.7%), 8/8. The cost of the added fold (≈ 180 ms) is included; the removals (≈ 250 ms) net out.

7 Register-resident round 2

Round 2’s message loop crossed the vector/general register-file boundary three times per output pair: the table-driven fold extracted its vector accumulator into a two-word struct; the message products $g_1 = a_1 b_1$, $g_\infty = (a_0 + a_1)(b_0 + b_1)$ went through the scalar field multiply (struct in, struct out); and the accumulate of Section 2 re-entered the vector file. The rewrite keeps the whole chain in vector registers: the fold returns its register unextracted; the products use an in-register Karatsuba multiply (five PMULLs — three for the product, two for the fold through $X^{128} \equiv X^7 + X^2 + X + 1$ — against the baseline’s six); the $\text{eq} \cdot g$ products feed the Section 2 accumulator directly. Per pair, the only remaining memory traffic is the four required stores of folded values and one eq load.

Measured: round-2 phase 358.1 $\rightarrow$ 311.6 ms (−13.0%), 8/8, every pair in $[-12.6\%, -13.3\%]$, taken on a machine with an active browser session (the min-of-5 protocol absorbs interactive bursts).

The rounds-3+ tail: fusing away a memory pass. The tail rounds (3 and beyond) apply the same treatment plus one thing round 2 had no opportunity for. The incumbent made

two passes per worker chunk: fold $a_{\text{in}}, b_{\text{in}}$ into $a_{\text{out}}, b_{\text{out}}$, then re-read the just-written multi-megabyte chunk to build the message — an L2/DRAM read of data that had been in vector registers moments earlier. The fused kernel folds each output pair in-register ($e + r(e + o)$) via `mul_q`, issues the one required store, and consumes the pair for the message while still in registers:

```

1: for  $x = 0 \dots lo - 1$  do
2:    $a_0, a_1, b_0, b_1 \leftarrow$  fold four input pairs at  $r$  ▷ in q registers
3:   store  $a_0, a_1, b_0, b_1$  ▷ the required write, once
4:    $g_1 \leftarrow a_1 b_1$ ;  $g_\infty \leftarrow (a_0 + a_1)(b_0 + b_1)$  ▷ mul_q, never leaving registers
5:    $acc_1 \oplus = eq_x \odot g_1$ ;  $acc_\infty \oplus = eq_x \odot g_\infty$  ▷ unreduced, §2
6: end for

```

The PMULL count is identical to the two-pass form; the entire gain is one traversal instead of two plus the removed struct crossings. Notably, two earlier attempts on this exact loop had failed (a memory-interfaced pair-multiply kernel, +10%; wide accumulators alone, 0%): they changed the arithmetic encoding and the accumulator while leaving the pass structure intact, which is where the cost actually lived.

Measured: rounds-3+ phase 449.5 $\rightarrow$ 306.6 ms (−31.8%), 8/8, every pair in [−31.1%, −32.6%] — the largest single win of the effort.

8 PCS open: composing the block scalar into the fold table

Shape for this section: the 2^{16} -compression open ($m = 30$, $L = 2^{23}$ packed slots, two batched claims). The ring-switch reduction leaves, for each claim k , an `eq` tensor in factored form $eq = eq_{\text{lo}} \otimes eq_{\text{hi}}$ (lengths B and L/B) and an $\mathbb{F}_2$ -linear map $L_k : \mathbb{F}_{2^{128}} \rightarrow \mathbb{F}_{2^{128}}$ (the γ_k -weighted byte-table fold). The *combine* pass materializes the batched basis

$$b[j] = \sum_k L_k(eq_{\text{lo}}[j \bmod B] \cdot eq_{\text{hi}}[\lfloor j/B \rfloor]), \quad j < L,$$

and was the largest bucket of the open: one field multiply plus one 16-load table fold per slot per claim, 58% of open time.

The multiply is removable. Multiplication by the block constant $e = eq_{\text{hi}}[\lfloor j/B \rfloor]$ is itself $\mathbb{F}_2$ -linear, so the per-slot map $G_{k,e}(w) = L_k(w \cdot e)$ is a composition of $\mathbb{F}_2$ -linear maps and is fully determined by its 128 monomial images $g_r = L_k(X^r e)$. Once per (claim, block), encode $G_{k,e}$ as a fresh 16×256 byte table: advance $X^r e$ by 127 exact shift-and-fold doublings (multiplication by X costs one shift and a conditional XOR of the reduction word), evaluate L_k on each via the existing table, and expand every byte position from its 8 generators by subset-sum doubling ($16 \cdot 255$ additions). About 4.3k word operations per block; the sweep then indexes only the composed table, and the per-slot multiply — $2L \approx 16.8\text{M}$ multiplies per open — is gone.

The catch is the block length. The balanced split used by the many-pass folds ($B = 2^{\lceil n/2 \rceil} = 2^{11}$ here) is too fine for the build to amortize; the deferred claims carry their own coarse split $B = \min(2^{15}, 2^{n-8})$, making `eq_lo` a single 512 KiB L2-resident stream shared by every block and the build ≈ 0.4 ops/slot. Correctness is exact, not approximate: every composed entry is an XOR combination of exact field products — the same XOR multiset as the slot-multiply path — so the transcript is bit-identical (unit equivalence test; verification of full proofs unchanged).

Measured (ST, minimum over samples): combine 94.4 $\rightarrow$ 78.5 ms, open total $\approx 160 \rightarrow \approx 145$ ms (−9%); at 8 threads the combine is 10.9 ms, a near-linear transfer. The idea was found

dormant in the challenge tree (present, never isolated or measured there) and re-derived.

Negative space for the open. (i) Three-way XOR fusion (EOR3) in the fold’s reduction tree is flat: LLVM already emits it under `target-cpu=native`, and the sweep is bound by its 32 loads per slot, not its XOR count. (ii) Fusing both claims into one sweep — two live composed tables, one store, no read-back of the intermediate — *regresses* 15%: 128 KiB of gather targets thrash L1, so the claim-sequential order stands. (iii) The round-1 lookahead coefficients accumulated in the combine’s tail are an accounting wash, not a speedup: tail-plus-initial-sumcheck costs 60.5 vs. 60.6 ms across the two trees’ opposite placements of the same work.

9 Summary

Optimization	Kind	Phase effect (ST)	Sign
§2 deferred reduction in registers	representation	round 2: −13.4%	d
§3 structurally-zero b rows	work removal	round 1: −2.9%	
§4 two-lane drain	ILP	round 1: −4.3%	
§5 unreduced accumulate, split weights	arithmetic	round 1: −9.1%	
§6 C side as stripe fold	structural	round 1: −5.7%	
§7 register-resident round 2	representation	round 2: −13.0%	
§7 fused rounds-3+ tail	representation + pass fusion	rounds 3+: −31.8%	
§8 composed fold table (PCS open)	work removal	open combine: −17%	d
round 1 cumulative		1477 → 1205 ms	(−)
round 2 cumulative (§2+§7)		433 → 312 ms	(−)
rounds 3+ cumulative		455 → 307 ms	(−)
PCS open cumulative (§8)		≈ 160 → ≈ 145 ms	(−)
end-to-end, all seven zerocheck changes		≈ 52,400 → ≈ 58,100 comp/s	(≈ +11%)

Negative space. The kept changes are one side of a 17-experiment record; the failures shaped them. On this core, re-encoding fixed work never paid: pairing multiplies through a memory-interfaced kernel (+10%), a four-register structured load (+12%, microcoded), three-way XOR fusion (+1.8%), an $8\times$ smaller gather table (−3.6% regression), and deferring an already-cheap reduction (0%) all failed, while every win either removed work, added independent work in flight, or removed register-file boundary crossings. In the commit-phase NTT, rewriting the butterfly multiply as a 3-PMULL Karatsuba with a q-resident interface regressed 58%: the generic butterfly already inlines the latency-optimal binus kernel, and six PMULLs with no repack beat three with one, in situ and not only in the microbenchmark. Two further structural transplants measured neutral end-to-end and were reverted despite making the zerocheck *bucket* look better (hoisting the challenge-independent prep under the commit; the lookahead double-round, which loses 7–15% in the zerocheck tail on both this machine and the reference tree’s, while *helping* the PCS open by 7% — the same technique, opposite signs, two call sites).